

# Smoothed Particle Hydrodynamics for Fluid Simulation

Krištof Zupan

Advanced Computer Graphics 2025/26, FRI, UL  
kz16876@student.uni-lj.si

---

## Abstract

*In this paper we discuss our implementation of Smoothed Particle Hydrodynamics (SPH) for a fluid simulation. Starting from a 2D CPU implementation and later adding GPU acceleration via the Taichi framework on Apple Metal, 3D support, and marching cubes for surface reconstruction. The CPU prototype is limited to around 100 particles at interactive rates, but the later GPU implementation enables simulation of up to 10 000 particles.*

---

## 1. Introduction

In this project we implemented a fluid simulation using Smoothed Particle Hydrodynamics (SPH). SPH tries to simulate a fluid with a collection of particles each carrying position, velocity, density and pressure. These values at any point in space are approximated by summing contributions from neighbouring particles weighted by a smoothing kernel.

We started with a 2D CPU implementation in Python to establish the algorithm as clearly as possible without any GPU-specific code. Afterwards we re-implemented it to the GPU using the Taichi framework with the Metal backend as the development was done on MacOS. After we got the parameters dialed in we extended the 2D simulation to 3D and added marching cubes for surface reconstruction to get a better visual result as was recommended in the project instructions. The implementation follows this papers from [IOS\*14], [HKK07] and also this repository [Lag23] that is based on a video implementation of the algorithm. The repository is available at: [www.github.com/kristofzupan/Smoothed-particle-hydrodynamics](https://www.github.com/kristofzupan/Smoothed-particle-hydrodynamics).

## 2. Implementation

### 2.1. SPH Algorithm

SPH approximates a fluid by kernel weighted sums over nearby particles. The density at particle  $i$  is estimated as

$$\rho_i = \sum_j m_j W_{ij}, \quad (1)$$

where  $m_j$  is the mass of particle  $j$ , and  $W$  is the smoothing kernel that has a smoothing radius defined for it [IOS\*14]. This is computed in `calculate_densities()` by iterating over all particle pairs.

**Pressure.** We convert density to pressure using a linear equation of

state

$$p_i = k(\rho_i - \rho_0), \quad (2)$$

where  $k$  is the pressure multiplier and  $\rho_0$  is the target rest density. When  $\rho_i > \rho_0$  the particle is in a dense region and pressure pushes it outward but when  $\rho_i < \rho_0$  it is in a low density region and pressure attracts the particles [HKK07]. This is implemented in `convert_density_to_pressure()`.

**Pressure force.** To guarantee symmetric forces we use the average of the two particles' pressures  $\bar{p}_{ij} = (p_i + p_j)/2$ , giving

$$\mathbf{F}_i^{\text{press}} = \sum_j m_j \frac{(p_i + p_j)}{2\rho_j} \nabla W_{ij}. \quad (3)$$

The gradient of  $W$  gives the direction and magnitude of the repulsion or attraction. This is computed in `calculate_pressure_force()`.

**Viscosity.** Viscosity smooths velocity differences between neighbouring particles. The force is defined by  $(v_j - v_i)$  weighted by a new viscosity kernel and a global viscosity constant that is defined at the start and lets us control how viscous the fluid is. We calculate this force in a very similar way as the above formula for pressure force [HKK07]. This is computed in `apply_viscosity()`.

**Smoothing kernels.** We use a spiky kernel for the pressure term and a Poly6 kernel for viscosity. Both return zero outside the radius  $h$ . The 2D pressure kernel is

$$W(r, h) = \frac{6(h-r)^2}{\pi h^4}, \quad (4)$$

implemented in `smoothing_kernel()` function. In 3D the kernel changes a lot to match the spherical volume integral and the same is true for the viscosity kernel [Lag23].

**Predicted positions.** To improve stability, densities and forces are evaluated at predicted positions  $\mathbf{x}_i = \mathbf{x}_i + \mathbf{v}_i \cdot \Delta t_{\text{pred}}$  with  $\Delta t_{\text{pred}} =$

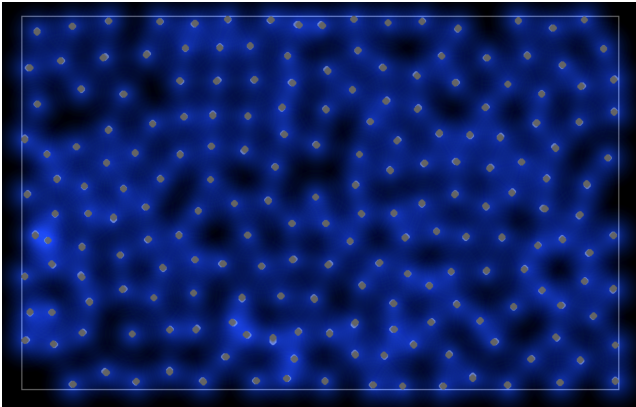
1/120s [IOS\*14] [Lag23]. This lets pressure react to where particles are heading rather than their current positions, which reduces instability at high velocities.

## 2.2. 2D CPU Implementation

The first implementation (`main.py`) runs entirely on the CPU using Python, NumPy and Pygame. It is intentionally simple and serves as the clearest possible reference for the algorithm. The simplified simulation pipeline is:

1. Apply gravity to velocity, compute predicted positions.
2. Compute particle densities with `calculate_densities()`.
3. Sum pressure and viscosity forces using `calculate_pressure_force()`.
4. Integrate velocities and positions, resolve boundary collisions via `resolve_collisions()`.

The first tests that we did beyond just testing a single particle falling due to gravity, was our visualization of how particle density is seen and behaves. Here we initialized the particles to random positions and looked at how the particles try to find enough space around them. We can see in the figure below how some parts are nicely separated and have constant density but the bottom left corner has some higher densities that resolved later in the simulation.

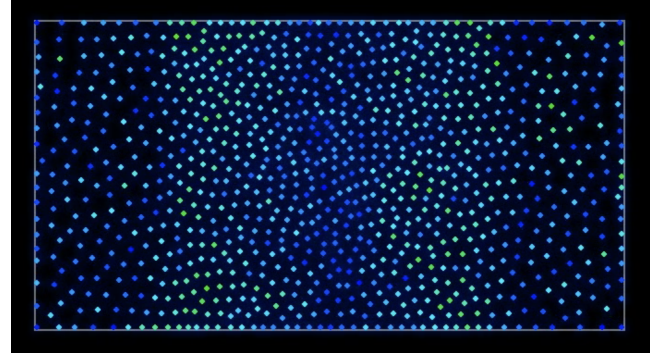


**Figure 1:** *Density visualization*

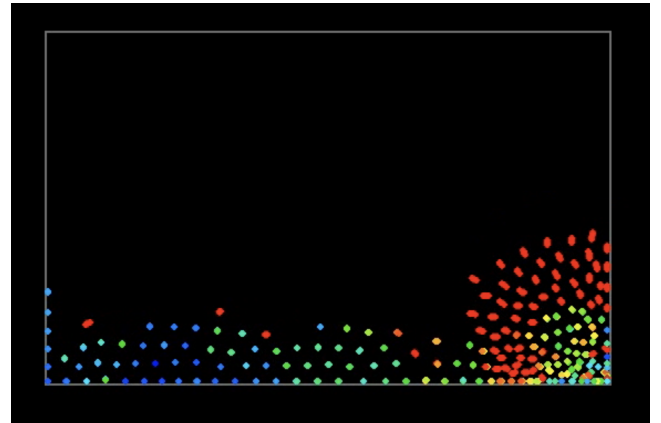
Afterwards the particles were initialized in a uniform grid arrangement inside a rectangular bounding box. Boundary collisions bounce the particle back of the walls with a damping factor of 0.9.

Here we also added visualization of particle speed mapped to a colour from blue (slow) to red (fast), making it easy to identify where the fluid is moving fastest. Mouse interaction is also supported: left-click attracts particles and right-click repels them within a configurable radius, which is useful for testing the simulation response and generally interacting with the fluid. Below we have two figures from this implementation. The first one does not have gravity enabled (set to 0) and the second one has gravity set to 10. Both were recorded at 30 fps but were not usable in real time.

The CPU implementation is limited to around 100 particles before the frame rate drops below interactive rates. The bottleneck



**Figure 2:** *CPU SPH without gravity*



**Figure 3:** *CPU SPH with gravity*

is the  $O(N^2)$  nested loop in `calculate_densities()` and `calculate_pressure_force()`, which is executed in plain Python with no optimization.

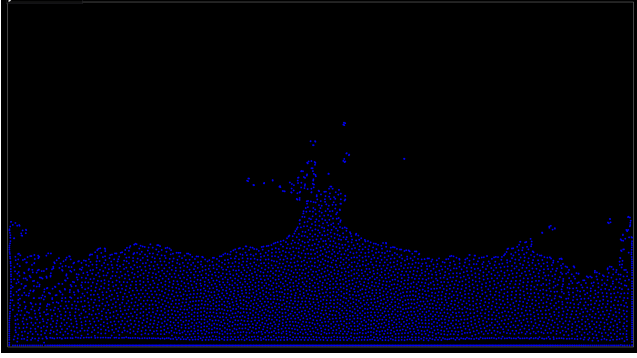
## 2.3. GPU Acceleration

To overcome the CPU bottleneck we re implemented the simulation to the GPU (`sph_gpu.py`) using the Taichi framework with the Apple Metal backend. Taichi allows writing GPU kernels in Python-like syntax. Functions are compiled to native GPU code at startup and the outer particle loop runs in parallel.

The flow change to a more centralized function calling in the main while loop of the simulation. One constraint of Taichi is that early returns inside kernel functions are not supported so we had to change a little the code to support this pattern, for example in the smoothing kernel we replace the early return with an if statement in the final return like seen below.

```
@ti.func
def smoothing_kernel(...):
    vol = (math.pi * (rad ** 4.0)) / 6.0
    val = ((rad - dist) ** 2.0) / vol
    return val if dist < rad else 0.0
```

With the GPU implementation we can simulate 10 000 particles comfortably in real time at 60FPS. This was enough to tune the parameters for visually convincing fluid behaviour: rest density  $\rho_0 = 18$ , pressure multiplier  $k = 500$ , viscosity  $\mu = 0.1$ , and smoothing radius  $h = 0.55$ . In the figure below you can see the results.



**Figure 4:** GPU SPH 5000 particles

#### 2.4. 3D SPH

We continued by converting the simulation to three dimensions (`sph_3d.py`). All particle arrays use 3-component Taichi vectors. The kernels are also updated as mentioned in Section 2.1 to account for the spherical volume in 3D. The bounding box is rendered as a wireframe by manually specifying its 12 edges, since Taichi does not have a built-in rectangle drawing call.

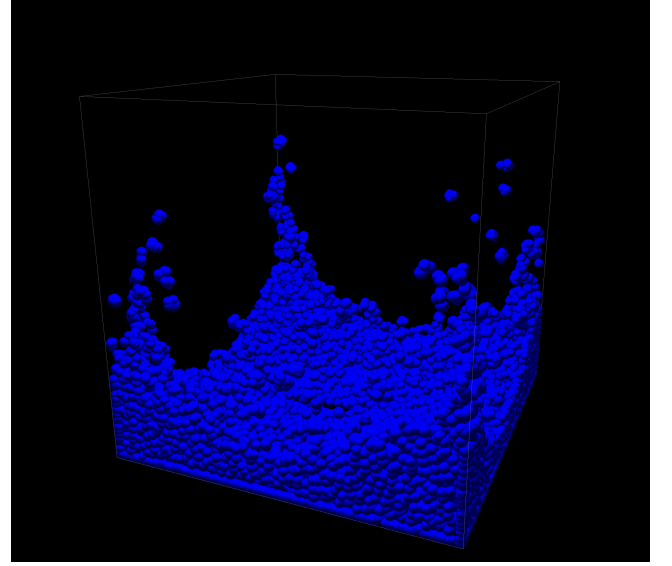
An orbit camera is implemented with left mouse button. Arrow keys control the zoom distance. This lets us inspect the fluid from any angle during the simulation.

The 3D simulation runs a little slower but can still manage 30 fps with 8000 particles. Parameters had to be re-adjusted to  $\rho_0 = 26$ ,  $h = 0.7$ ,  $\mu = 0.05$ , and 5 substeps per frame. The higher rest density is needed because in 3D each particle has more neighbours within the same smoothing radius, so the raw density values are larger than in 2D. The results can be seen in the figures 5 and 6 below.

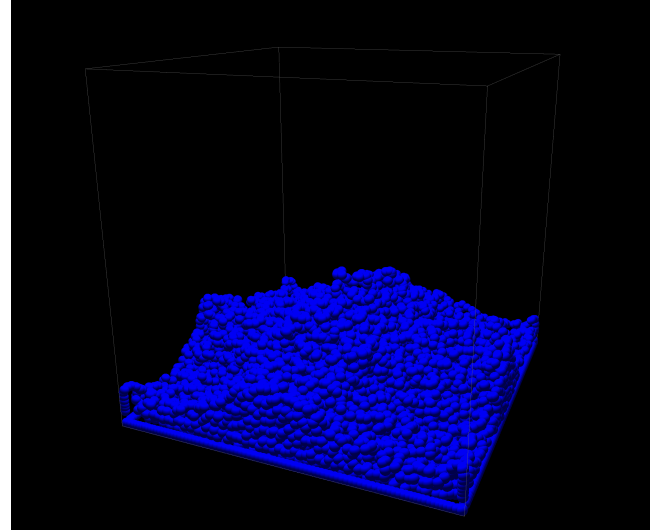
#### 2.5. Surface Reconstruction with Marching Cubes

Rendering individual particles is sufficient for simulations but as fluid rendering we need a better approach. To produce a smooth continuous fluid surface we added marching cubes reconstruction (`sph_mc.py`). After each simulation frame we trigger `build_mc()` that samples a density field on a  $64^3$  regular grid covering the whole simulation bounding box (we set the resolution to 64).

The borders of the density grid are clamped to zero so the extracted surface always closes at the boundary as we had problems with transparent sides of the fluid in the bounding box. The mesh at density = 20 is then extracted on the CPU by `skimage.measure.marching_cubes` in `extract_surface()`, which also computes per-vertex normals.



**Figure 5:** 3D SPH after the initial grid splash



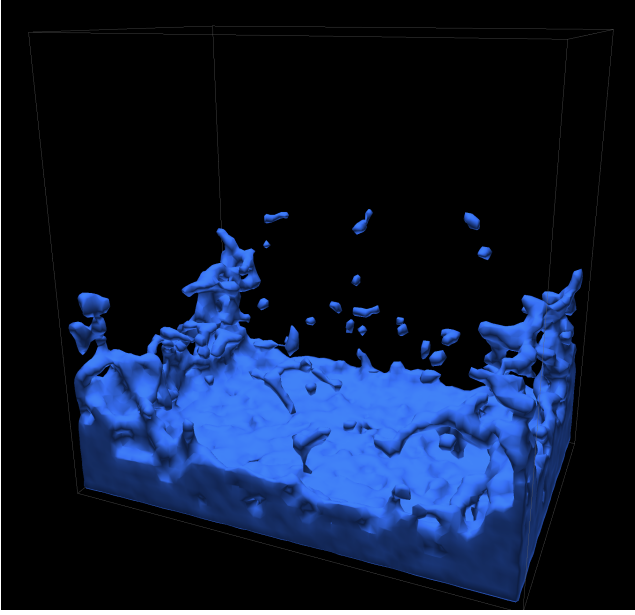
**Figure 6:** 3D SPH after the initial grid settles

The resulting indexed mesh is flattened to a flat triangle list and uploaded to Taichi's scene renderer each frame via `scene.mesh()`.

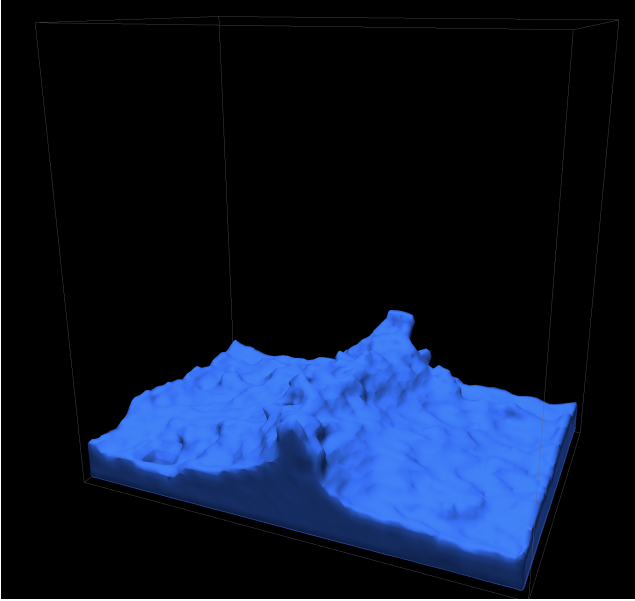
The main parameter to tune is the iso value. Values that are too low produce an overly thick blob that hides the fluid dynamics but too high and the surface breaks up into thin separate blobs.

### 3. Results

Figures 7 and 8 show the final marching cubes GPU simulation (8192 particles). The first figure shows the simulation after the initial grid of particles hits the ground and splashes to the sides. The second figure shows the simulation a little later after the fluid is



**Figure 7:** *Marching Cubes SPH after the initial grid splash*



**Figure 8:** *Marching Cubes SPH after the initial grid settles*

settling and there is a wave in the middle. In the end we were very happy with the resulting fluid simulation as they look like a good fluid simulation. There definitely are a lot of possible improvements. In SPH the particles do not come together to form realistic droplets (while researching we found a possible improvement by adding another near pressure force to account for that). The marching cubes visualization approach could also be improved with ray marching as a possible future improvement.

#### 4. Performance

All GPU measurements were taken on an Apple M4-series GPU. Table 1 reports measured frame rates for varying particle counts across all implementations.

**Table 1:** *Measured performance.*

| Implementation          | Particles | FPS  |
|-------------------------|-----------|------|
| CPU (main.py)           | 100       | 30   |
| CPU (main.py)           | 200       | 9    |
| GPU 2D (sph_gpu.py)     | 1 000     | ~400 |
| GPU 2D (sph_gpu.py)     | 5 000     | 160  |
| GPU 2D (sph_gpu.py)     | 10 000    | 72   |
| GPU 3D (sph_3d.py)      | 5 000     | 112  |
| GPU 3D (sph_3d.py)      | 10 000    | 42   |
| GPU 3D + MC (sph_mc.py) | 500       | 48   |
| GPU 3D + MC (sph_mc.py) | 1000      | 24   |

The main bottleneck is the  $O(N^2)$  all pairs interaction loop for density and force computation. The GPU parallelises the outer loop over particles, giving a large speedup over the CPU version, but this search still limits the feasible particle count at real-time rates. The marching cubes step adds an additional cost for resolution 64 that is manageable but limits how fine the density grid can be made without dropping below real time frame rates.

#### 5. Conclusion

In this project we implemented a complete SPH fluid simulation pipeline. We started from a readable 2D CPU prototype that clearly shows the algorithm, then added GPU acceleration via Taichi and Apple Metal to enable real time simulation of thousands of particles. Extending to 3D required adjusting the kernels and the simulation parameters. Finally, marching cubes surface reconstruction significantly improves the visual quality compared to rendering individual particles.

The main remaining limitation is the  $O(N^2)$  complexity of particle interactions. Replacing the all pairs search loop with a spatial hash grid would reduce this to  $O(N)$  average cost and would be the most impactful improvement for scaling to larger particle counts.

#### References

- [HKK07] HARADA T., KOSHIZUKA S., KAWAGUCHI Y.: Smoothed particle hydrodynamics on GPUs. [www.inf.ufrgs.br/cgi2007/cd\\_cgi/papers/harada.pdf](http://www.inf.ufrgs.br/cgi2007/cd_cgi/papers/harada.pdf). 1
- [IOS\*14] IHMSEN M., ORTHMANN J., SOLENTHALER B., KOLB A., TESCHNER M.: SPH fluids in computer graphics. *Eurographics 2014 – State of the Art Reports* (2014). [www.cgl.ethz.ch/Downloads/Publications/Papers/2014/Sol14a/Sol14a.pdf](http://www.cgl.ethz.ch/Downloads/Publications/Papers/2014/Sol14a/Sol14a.pdf). doi: 10.2312/egst.20141034. 1, 2
- [Lag23] LAGUE S.: Fluid-sim, 2023. <https://github.com/SebLague/Fluid-Sim/tree/Episode-01>. 1, 2